

Simulación de Partículas

CUDA + OpenGL

Computación paralela en GPU con renderizado en tiempo real.

Emily Jiménez & Héctor López



Índice

01

Objetivos técnicos y de rendimiento

Metas de implementación y métricas de evaluación

02

Arquitectura del sistema

Cómo CUDA y OpenGL comparten recursos en la GPU

03

Implementación CUDA

Kernels de física, gestión de memoria

04

Renderizado con OpenGL

El pipeline gráfico: de coordenadas 2D a imagen en pantalla

05

Resultados y rendimiento

Benchmarks, análisis, conclusiones

Objetivos técnicos y de rendimiento

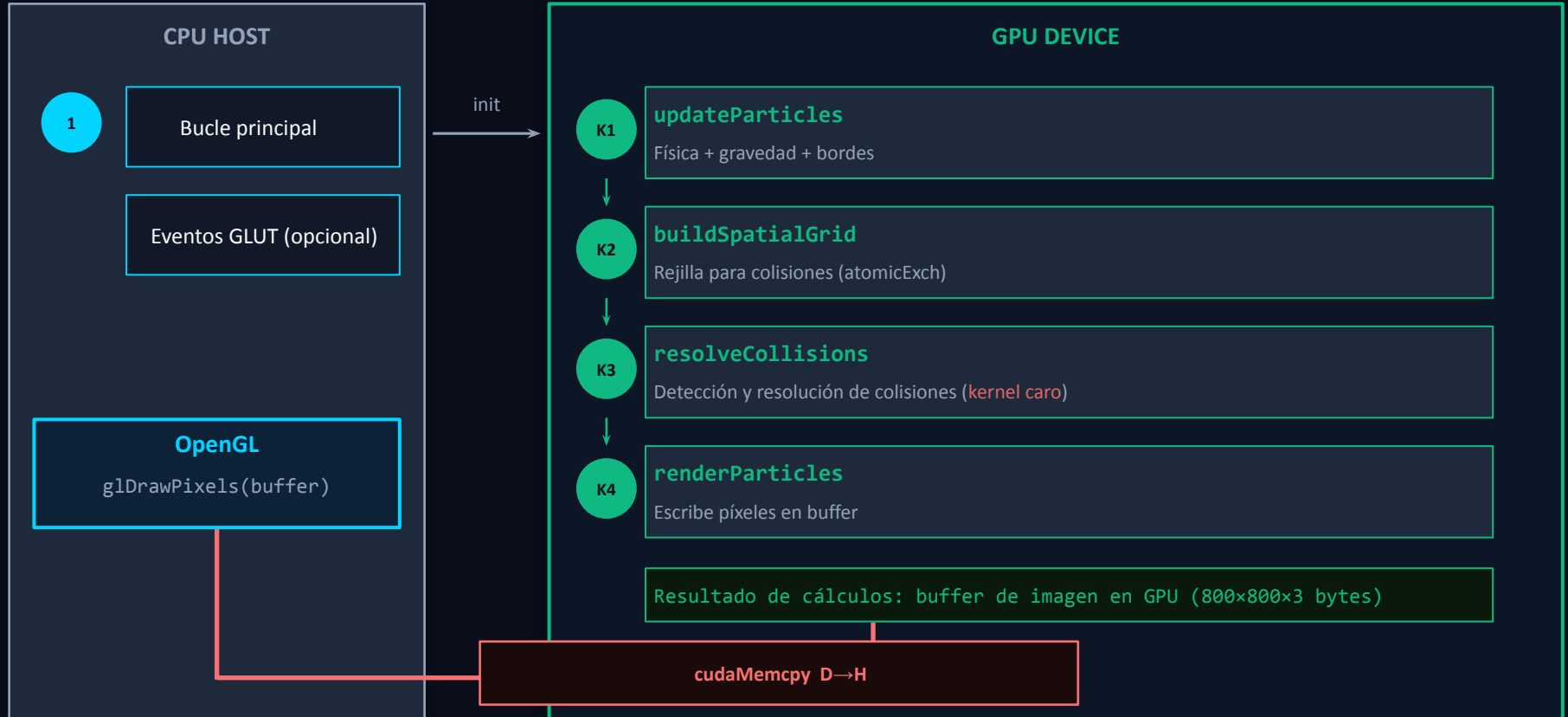
Técnicos

- Realizar simulación de partículas en tiempo real. Aprovechando GPU para computo y renderizado.
- Lanzar 1 hilo por partícula.
- Usaremos un grid para detectar colisiones eficientemente.
- Renderizar en GPU una vez por frame.
- Hacer una versión en CPU.

Rendimiento

- Conseguir un rendimiento (FPS) estable con 80.000 partículas.
- Analizar el coste de transferencia entre GPU y CPU por cada frame.
- Analizar los tiempos de los distintos kernels.
- Hacer una comparativa de CPU vs GPU.

Arquitectura del sistema



Implementación CUDA (I)

Kernels K1 y K2 — física de partículas y construcción de rejilla espacial

K1 updateParticles

```
__global__ void updateParticles(Particle* p, float dt, float g) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i >= N) return;

    p[i].vel.y -= g * dt;          // gravedad
    p[i].pos.x += p[i].vel.x * dt; // posicion
    p[i].pos.y += p[i].vel.y * dt;

    if (p[i].pos.x < 0) {p[i].pos.x=0; p[i].vel.x*=-0.6f;}
    if (p[i].pos.x>WIDTH) {p[i].pos.x=WIDTH; p[i].vel.x*=-0.6f;}
    if (p[i].pos.y < 0) {p[i].pos.y=0; p[i].vel.y*=-0.6f;}
    if (p[i].pos.y>HEIGHT) {p[i].pos.y=HEIGHT;p[i].vel.y*=-0.6f;}
}
```

Física + gravedad + rebote con bordes

1 hilo por partícula (cálculo clásico de índice de CUDA)

Cada frame, la partícula avanza según su velocidad actual

Rebote con bordes

K2 buildSpatialGrid

```
__global__ void buildSpatialGrid(const Particle* p, int* cellHeads, int* next)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i >= N) return;
    int cell = (int)(p[i].pos.y/CELL_SIZE)*GRID_W + (int)(p[i].pos.x/CELL_SIZE);
    next[i] = atomicExch(&cellHeads[cell], i); // insercion atomica
}
```

Rejilla espacial para detección eficiente de colisiones

Celdas 8x8 px → evita $O(N^2)$ comparaciones

atomicExch: inserción sin condiciones de carrera

Implementación CUDA (II)

Kernels K3 y K4 — resolución de colisiones (~93% del frame) y renderizado

K3 **resolveCollisions**

```
__global__ void resolveCollisions(
    const Particle* snap, Particle* p,
    const int* cellHeads, const int* next, float rest) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i >= N) return;
    Particle self = snap[i]; // snapshot: solo lectura

    for (int dy=-1; dy<=1; dy++) { // 9 celdas vecinas
        for (int dx=-1; dx<=1; dx++) {
            for (int j=cellHeads[cell]; j!=-1; j=next[j]) {
                float dist = distance(self, snap[j]);
                if (dist < COLLISION_DISTANCE)
                    applyImpulse(&self, snap[j], rest);
            }
        }
        p[i] = self; // escribe resultado en d_particles
    }
}
```

Kernel más costoso — ~93% del tiempo de frame

— snap[] solo lectura — todos los hilos ven el mismo estado

— 9 celdas vecinas → $O(N \times \text{densidad})$, no $O(N^2)$

— Impulso elástico proporcional a velocidad relativa

— Escribe en p[] (d_particles) → resultado del frame

K4 **renderParticles**

```
__global__ void renderParticles(Particle* p, Pixel* img) {
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i >= N) return;
    int idx = (int)p[i].pos.y * WIDTH + (int)p[i].pos.x;
    img[idx] = {p[i].color.x*255, p[i].color.y*255, p[i].color.z*255};
}
```

Escribe píxeles directamente en d_img en GPU

— $O(1)$ por hilo — escritura directa a buffer GPU

Renderizado con OpenGL

K4 renderParticles (CUDA)

Cada hilo escribe el color de su partícula en el píxel correspondiente de **d_img**, un buffer que vive en memoria de GPU.

```
img[y * WIDTH + x] = {r, g, b}
```

cudaMemcpy D→H (~1.5% del frame)

El buffer **d_img** se copia de GPU a **h_img** en CPU. Transferencia rápida, siempre de mismo tamaño.

```
cudaMemcpy(h_img, d_img,  
WIDTH*HEIGHT*sizeof(Pixel),  
cudaMemcpyDeviceToHost)
```

glDrawPixels (OpenGL)

OpenGL vuelca **h_img** directamente al framebuffer píxel a píxel. No hay shaders ni geometría — OpenGL actúa como visor.

```
glDrawPixels(WIDTH, HEIGHT, GL_RGB,  
GL_UNSIGNED_BYTE, h_img)
```

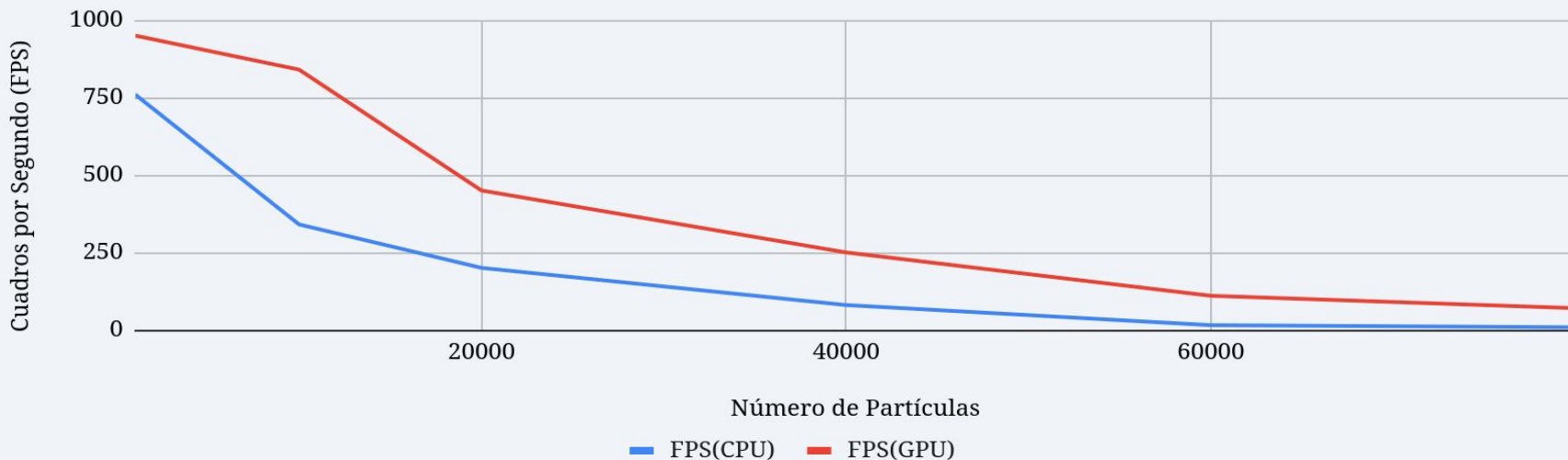
glutSwapBuffers

Intercambia los dos buffers de GLUT. El buffer recién dibujado aparece en pantalla, el otro queda listo para el siguiente frame. Evita parpadeo

```
glutSwapBuffers()
```

Resultados y rendimiento

Rendimiento: CPU vs GPU según el Número de Partículas



1.25x

Speedup vs CPU 1000 particulas

2.25x

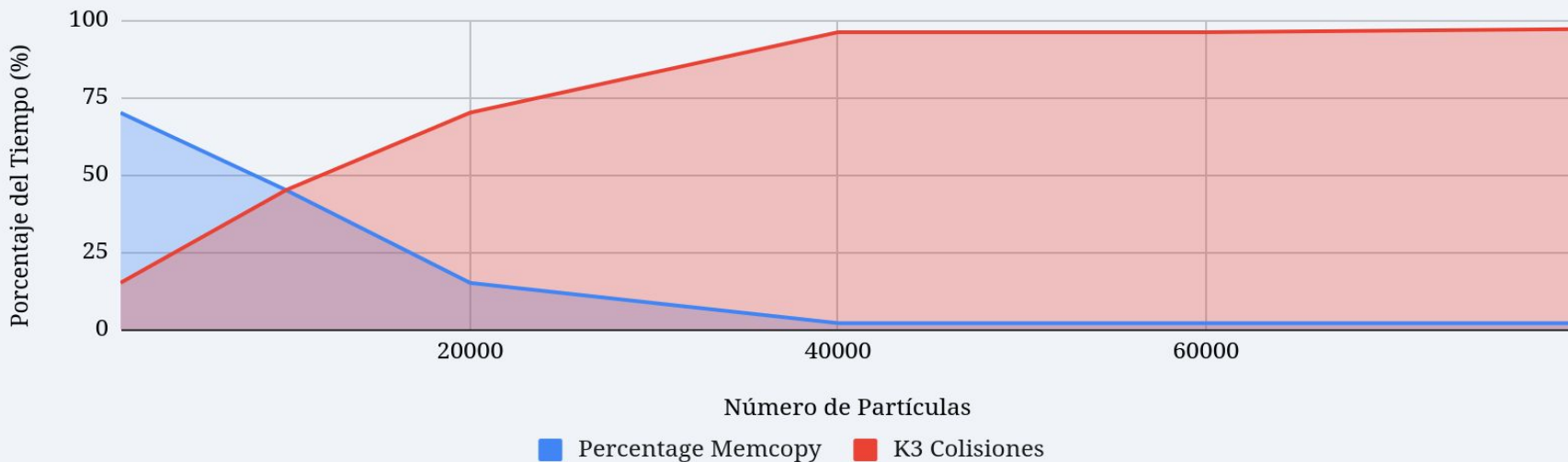
Speedup vs CPU 20000 particulas

8.75x

Speedup vs CPU 80000 particulas

Resultados y rendimiento

Distribución de Carga: Memcopy vs Colisiones



0.21x

K3 Colisiones / Memcopy 1000 partículas

4.66x

K3 Colisiones / Memcopy 20000 partículas

48.5x

K3 Colisiones / Memcopy 80000 partículas

Resumen de optimizaciones

- Un hilo por partícula en los 4 kernels: máximo paralelismo, sin dependencias entre hilos en K1, K2 y K4.
- Rejilla espacial 8×8px: cada partícula sólo compara con 9 celdas vecinas, evitando $O(N^2)$ comparaciones.
- Partículas suben a GPU una sola vez (al principio).
- Única transferencia D→H es la imagen (~1.9MB) → no las partículas, que permanecen en GPU. Así CPU recibe imagen lista.
- 256 hilos por bloque: configuración que maximiza la ocupación del SM.
- Accesos coalescentes a memoria: hilos consecutivos acceden a posiciones de memoria coalescentes.
- OpenGL dibuja pixeles directamente: sin shaders ni pipeline gráfico complejo.

Conclusiones

- El paralelismo masivo de GPU es imprescindible para simulaciones.
- La rejilla es la optimización más crítica en este proyecto. $O(N^2) \rightarrow O(N \times \text{densidad local})$
- La arquitectura modular (kernel de física + pipeline gráfico) es fácilmente extensible.

Video de simulación: <https://youtu.be/az2ncl-vh9I>